ANP-智能体通信元协议规范（Draft）

- 文档编号：ANP-06
- 标题：ANP-智能体通信元协议规范
- 状态：Draft
- 版本：1.2
- 语言：中文
- 适用范围：本规范适用于 ANP 智能体之间的语义元协议协商、接口选择、Profile 选择、安全模式选择、Schema 选择与协商结果复用。

备注：

- 元协议不是智能体协作过程中的必需协议，而是可选协议。正常情况下，智能体身份、消息、发现、描述等协议已经可以满足大多数互操作需求；元协议主要用于需要动态协商两个智能体之间独有接口、临时协议或特殊执行约束的场景。
- 本规范当前仍处于草案状态，尚未发布；其目标是把早期消息内元协议协商模型升级为与当前 ANP 描述、发现、Core Binding、DID 身份认证和消息 Profile 体系兼容的独立协商 Profile。
- 本规范不修改 ANP-07 智能体描述协议；本文定义的 `MetaProtocolInterface` 是 ANP-06 定义的扩展接口类型，可以声明在 Agent Description 的 `interfaces` 数组中。
- 新实现应使用本规范定义的 `anp.meta.negotiation.v1` 与 `anp.negotiate`。早期基于加密消息内部二进制 `PT=00` 的协商方式已经废弃，本规范不要求也不建议兼容旧实现。

摘要

元协议（Meta-Protocol）是“协商通信协议的协议”。在智能体网络中，它用于让调用方和目标智能体根据意图、能力、约束、安全要求和可用接口，选择后续真正执行业务交互时应使用的协议、接口、Profile、Schema、内容类型和执行模式。

本规范将 ANP 元协议重新定义为一个独立的语义协商 Profile：

```
Agent Discovery
  -> Agent Description
  -> MetaProtocolInterface
  -> anp.get_capabilities
  -> anp.negotiate
  -> selected interface / profile / security profile / schema
  -> business call or message exchange
```

与早期“在消息负载中用二进制 Protocol Type 区分元协议、应用协议、自然语言协议和验证协议”的设计不同，本规范不定义新的传输层、不定义新的加密消息格式，也不替代 ANP Core Binding。元协议协商消息应作为普通 ANP JSON-RPC 请求运行在现有 ANP 端点上。

1. 背景与问题

智能体之间的互操作通常先依赖公开描述：目标智能体通过 Agent Description 暴露能力、接口和安全要求，调用方读取这些描述后选择合适的调用方式。随着智能体和接口类型增多，仅靠静态描述会遇到以下问题：

1. **静态描述无法覆盖运行时变化**：服务端限流、可用安全模式、临时关闭的接口、用户授权状态和部署策略都可能动态变化。
2. **同一意图可能有多个可用接口**：例如酒店预订既可以通过结构化 OpenRPC 接口完成，也可以通过自然语言消息接口完成。
3. **调用方能力也影响选择**：调用方是否支持某个 ANP Profile、某种安全模式、某种内容类型或异步执行模式，会影响最终协议选择。
4. **自然语言临时协商成本高**：完全依赖自然语言多轮协商可以提高灵活性，但会增加延迟、成本和不确定性。
5. **早期消息内协商模型与当前协议栈脱节**：早期 ANP-06 直接扩展加密消息负载，并使用二进制 `PT` 字段区分协议类型；当前 ANP 已经收敛到 DID 发现、Agent Description、`ANPMessageService`、JSON-RPC 2.0 Core Binding 和运行时能力确认。

因此，新的 ANP-06 把元协议定位为"语义协商控制面"：它不承载业务数据，而是帮助双方在业务执行前确定应使用哪一种互操作路径。

1.1 设计来源

ANP 元协议的设计参考和借鉴了 Agora Protocol（论文：*A Scalable Communication Protocol for Networks of Large Language Models*），尤其是通过元协议协商智能体通信方式、结合自然语言的灵活性与结构化协议效率的思路。ANP 将这些思路与自身的智能体描述、发现、身份认证和 Core Binding 架构结合；具体协商接口和互操作要求由本规范定义。

2. 设计目标与非目标

2.1 设计目标

本规范的目标是：

- 定义一个标准 Profile：`anp.meta.negotiation.v1`；
- 定义一种可声明在 Agent Description `interfaces` 数组中的 `MetaProtocolInterface`；
- 复用 ANP Core Binding 的 JSON-RPC 2.0、`params.meta/body/auth`、`profile`、`security_profile`、`target` 和错误模型；
- 复用 `anp.get_capabilities` 作为运行时能力确认方法；
- 定义 `anp.negotiate` 方法，用于基于调用方意图、能力和约束选择最终执行接口；
- 支持结构化接口优先、自然语言 fallback、缓存复用和未来协议 artifact 发布；
- 明确早期 `PT=00` 消息协商模型已经废弃，且不属于本规范兼容范围。

2.2 非目标

本规范不定义：

- 新的传输层、会话层或二进制消息头；
- 新的 DID 方法或新的身份认证机制；
- 新的端到端加密算法；
- `anp.get_capabilities` 的完整规范；该方法由 Core Binding 定义；
- 业务接口本身的请求、响应和状态机；

- 强制 AI 代码生成、远程代码加载或代码交换流程；
- 全网协议注册表、共识协议选举算法或经济激励机制。

3. 与现有 ANP 协议的关系

3.1 与 Agent Description 的关系

Agent Description 是智能体的公开入口，描述智能体的 DID、所有者、信息资源、接口和安全配置。本文定义的 `MetaProtocolInterface` 是一种可放入 Agent Description `interfaces` 数组中的扩展接口类型，用于声明：

- 该智能体支持 ANP-06 元协议协商；
- 协商端点在哪里；
- 端点使用哪个 binding；
- 支持哪些协商方法；
- 可协商哪些对象，例如 Profile、Interface、Schema、Security Profile、Content Type 和 Execution Mode。

本规范不要求修改 ANP-07。实现者可以在保持 Agent Description 现有结构的前提下，将 `MetaProtocolInterface` 作为扩展 `Interface` 对象发布。

3.2 与 Agent Discovery 的关系

发现流程仍由 ANP-08 定义。调用方可以先通过：

```
https://{domain}/.well-known/agent-descriptions
```

获得 Agent Description URL，再读取目标智能体描述文档，并从 `interfaces` 中查找 `MetaProtocolInterface`。

3.3 与 Core Binding 的关系

ANP-06 的运行时消息使用 ANP Core Binding 约定：

- JSON-RPC 2.0；
- `params.meta`、`params.auth`、`params.body` 结构；
- `meta.profile` 表示本次请求的解释 Profile；
- `meta.security_profile` 表示本次请求采用的安全模式；
- `meta.target` 表示目标建模；
- JSON-RPC `error` 对象承载错误。

`anp.negotiate` 的 `params.meta.profile` MUST 等于：

```
anp.meta.negotiation.v1
```

3.4 与 DID Document 和 ANPMessageService 的关系

Agent Description 中的 `MetaProtocolInterface.url` 是静态声明和可用入口 hint。身份和服务发现的权威仍来自 DID Document 中公开的 `ANPMessageService`，运行时能力的权威来自 `anp.get_capabilities` 返回结果。

当 Agent Description、DID Document 和运行时能力结果存在冲突时，调用方 SHOULD 按以下顺序处理：

1. 使用 DID Document 和 DID 解析结果建立身份与服务端点信任；
2. 调用 `anp.get_capabilities` 获取运行时能力；
3. 将 Agent Description 中的 `MetaProtocolInterface` 作为发现和语义 hint；
4. 若静态 hint 与运行时能力冲突，调用方 MUST 以运行时能力结果为准，并刷新本地缓存。

3.5 与 DID:WBA 和安全机制的关系

本规范不重新定义 DID 方法或身份认证。通用请求认证、HTTP Message Signatures 与 access token 遵循 ANP-02；WBA 方法验证遵循 ANP-03，其他 DID 按各自方法验证。`auth.origin_proof` 与消息服务间校验遵循相关消息 P1/P2 及所属 Profile 的安全要求。文档版本 1.2 不改变本规范的草案状态或 `anp.meta.negotiation.v1` wire 标识。

`anp.negotiate` 的协商结果不是身份凭证、授权凭证、access token、Verifiable Credential 或人类授权结果。

4. Profile 标识与最小互通

4.1 Profile 名称

本规范定义的 Profile 名称是：

```
anp.meta.negotiation.v1
```

Agent Description、`anp.get_capabilities` 返回值、`params.meta.profile` 和协商结果中的 Profile 引用均应使用该名称。

4.2 标准方法

本规范定义一个标准方法：

```
anp.negotiate
```

该方法用于根据调用方 intent、调用方能力、候选接口和约束，返回后续业务交互应使用的能力、接口、Profile、安全模式、内容类型、Schema 和执行模式。

4.3 最小互通要求

一个支持 `anp.meta.negotiation.v1` 的实现至少 MUST 支持：

1. 在运行时能力中声明 `anp.meta.negotiation.v1`；
2. 在支持元协议协商的端点上处理 `anp.negotiate`；
3. 识别 `MetaProtocolInterface` 的基本字段：`type`、`profile`、`binding`、`url`、`methods`；
4. 支持 `transport-protected` 安全模式下的协商；

5. 返回结构化 `NegotiationResult`；

6. 对不支持的 Profile、接口、安全模式或内容类型返回明确错误；

7. 不把协商结果解释为业务执行授权。

5. MetaProtocolInterface 声明

5.1 语义

`MetaProtocolInterface` 表示一个智能体公开的语义元协议协商接口。它是 Agent Description `interfaces` 数组中的一种扩展接口类型。

该接口负责说明：

- 调用方应向哪个 URL 发送协商请求；
- 协商请求使用什么 binding；
- 支持哪些协商方法；
- 可以协商哪些对象；
- 需要哪些安全配置。

5.2 字段定义

字段	类型	要求	说明
`id`	string	SHOULD	Interface 的稳定标识，用于协商结果引用。
`type`	string	MUST	固定为 `MetaProtocolInterface`。
`protocol`	string	SHOULD	建议为 `ANP`，表示这是 ANP 协议接口。
`version`	string	SHOULD	接口声明版本。
`profile`	string	MUST	固定为 `anp.meta.negotiation.v1`。
`binding`	string	MUST	本版本应为 `jsonrpc-2.0`。
`url`	string	MUST	协商端点 URL。该 URL 是静态 hint，运行时仍应以 DID Document 和 `anp.get_capabilities` 为准。
`methods`	array	MUST	至少包含 `anp.negotiate`，通常也包含 `anp.get_capabilities`。
`security`	array/string	MAY	引用 Agent Description 的安全定义。
`securityProfiles`	array	SHOULD	支持的安全模式，例如 `transport-protected`、`direct-e2ee`。
`negotiates`	array	SHOULD	可协商对象列表。
`inputSchema`	string	MAY	协商请求 Schema 的 URI。
`outputSchema`	string	MAY	协商结果 Schema 的 URI。

字段	类型	要求	说明
`description`	string	SHOULD	人类可读说明。

5.3 可协商对象

`negotiates` 中的常见值包括：

值	说明
`profiles`	协商后续业务调用使用的 ANP Profile。
`interfaces`	协商后续使用哪个 Agent Description interface。
`schemas`	协商请求和响应 Schema。
`security_profiles`	协商安全模式。
`content_types`	协商内容类型。
`execution_modes`	协商同步调用、消息、异步任务、流式或自然语言 fallback 等执行模式。
`protocol_artifacts`	协商可复用协议 artifact、digest 或 URI。

5.4 声明示例

```
{
  "id": "interface.negotiation.default",
  "type": "MetaProtocolInterface",
  "protocol": "ANP",
  "version": "1.0",
  "profile": "anp.meta.negotiation.v1",
  "binding": "jsonrpc-2.0",
  "url": "https://grand-hotel.com/anp",
  "methods": [
    "anp.get_capabilities",
    "anp.negotiate"
  ],
  "security": ["didwba_sc"],
  "securityProfiles": [
    "transport-protected"
  ],
  "negotiates": [
    "profiles",
    "interfaces",
    "schemas",
    "security_profiles",
    "content_types",
    "execution_modes"
  ],
  "description": "Semantic meta-protocol negotiation interface for selecting the best
execution interface."
```

```
}
```

5.5 推荐端点部署方式

实现方 SHOULD 优先复用统一 ANP endpoint，例如：

```
https://example.com/anp
```

该端点可以同时支持：

```
anp.get_capabilities
anp.negotiate
direct.send
group.send
...
```

实现方 MAY 为元协议协商部署独立端点，例如：

```
https://example.com/anp/negotiation
```

但这只是部署选择，不是协议强制要求。无论 URL 如何组织，服务身份和运行时能力仍应按 DID Document 与 `anp.get_capabilities` 进行确认。

6. 协商流程

6.1 总体流程

```
Caller Agent
  -> discover Agent Description
  -> parse MetaProtocolInterface
  -> resolve target DID and ANPMessageService
  -> call anp.get_capabilities
  -> call anp.negotiate
  -> receive NegotiationResult
  -> invoke selected interface or profile
```

6.2 Step 1：发现 Agent Description

调用方通过 ANP-08 主动发现或其他可信渠道获得目标 Agent Description URL。

6.3 Step 2：解析 MetaProtocolInterface

调用方读取 Agent Description，并在 `interfaces` 数组中查找：

```
{
  "type": "MetaProtocolInterface",
  "profile": "anp.meta.negotiation.v1"
}
```

如果目标 Agent 没有声明 `MetaProtocolInterface`，调用方 MAY：

- 直接使用 Agent Description 中已有的结构化接口；
- 使用自然语言接口；
- 终止自动协商并请求人工处理；
- 使用双方已缓存的协议 artifact。

6.4 Step 3：运行时能力确认

调用方 SHOULD 在首次交互前调用 `anp.get_capabilities`。该方法返回目标端点当前支持的 Profile、安全模式、内容类型和限制。

`anp.get_capabilities` 可以匿名调用；若服务端基于调用方身份返回不同能力集合，调用方 MAY 在完成认证后再次调用。

6.5 Step 4：语义协商

调用方发送 `anp.negotiate` 请求，携带：

- 调用方 intent；
- 所需能力；
- 调用方支持的 Profile、安全模式和内容类型；
- 候选 interface 引用；
- 延迟、授权、自然语言 fallback 等约束。

目标智能体根据静态描述、运行时能力、本地策略、授权状态和调用方能力，返回 `NegotiationResult`。

6.6 Step 5：执行业务交互

收到 `NegotiationResult` 后，调用方按照 `result.selected` 和 `result.execution` 执行真正业务调用或消息交互。

`anp.negotiate` 的响应不是业务响应。业务接口是否成功、是否需要人类授权、是否产生交易或状态变更，由后续被选择的业务协议决定。

7. `anp.negotiate` 方法

7.1 方法语义

`anp.negotiate` 是一个 JSON-RPC 2.0 Request 方法，用于执行语义元协议协商。

方法名：

```
anp.negotiate
```

请求的 `params.meta.profile` MUST 等于：

```
anp.meta.negotiation.v1
```

目标建模模式通常为 `agent-addressed`。当某个部署把元协议协商作为 service-scoped 控制面能力暴露时，具体 Profile 或实现文档 MUST 明确声明目标绑定规则。

7.2 请求结构


```
{
  "jsonrpc": "2.0",
  "id": "req-neg-001",
  "method": "anp.negotiate",
  "params": {
    "meta": {
      "profile": "anp.meta.negotiation.v1",
      "security_profile": "transport-protected",
      "sender_did": "did:wba:user.example.com:agents:personal-assistant:e1_example",
      "target": {
        "kind": "agent",
        "did": "did:wba:grand-hotel.com:service:hotel-assistant:e1_example"
      },
      "operation_id": "op-neg-001",
      "created_at": "2026-06-27T12:00:05Z",
      "content_type": "application/json"
    },
    "auth": {
      "origin_proof": {
        "type": "anp-rfc9421-origin-proof-v1",
        "keyid": "did:wba:user.example.com:agents:personal-assistant:e1_example#key-1",
        "contentDigest": "sha-256=:BASE64URL_DIGEST:",
        "signature": "BASE64URL_SIGNATURE"
      }
    },
    "body": {
      "negotiation_id": "neg-20260627-001",
      "mode": "structured_selection",
      "intent": {
        "name": "book_hotel_room",
        "description": "Book a hotel room for two people next Friday.",
        "intentTags": [
          "hotel.booking",
          "reservation.create"
        ]
      },
      "requiredCapabilities": [
        "cap.hotel.booking"
      ],
      "callerCapabilities": {
        "supportedProfiles": [
          "anp.core.binding.v1",
          "anp.direct.base.v1",
          "anp.rpc.v1"
        ],
        "supportedSecurityProfiles": [
          "transport-protected",
```

```
          "direct-e2ee"
        ],
        "supportedContentTypes": [
          "application/json",
          "text/plain"
        ]
      },
      "constraints": {
        "preferredInterfaceTypes": [
          "StructuredInterface",
          "NaturalLanguageInterface"
        ],
        "requiresHumanAuthorization": true,
        "maxLatencyMs": 3000,
        "allowNaturalLanguageFallback": true
      },
      "candidateInterfaceRefs": [
        "interface.booking.structured.v1",
        "interface.conversation.nl.v1"
      ]
    }
  }
}
```

7.3 `body` 字段定义

字段	类型	要求	说明
`negotiation_id`	string	SHOULD	协商过程标识。
`mode`	string	SHOULD	协商模式，默认 `structured_selection`。
`intent`	object	MUST	调用方希望完成的意图。
`requiredCapabilities`	array	MAY	调用方认为必须满足的能力标识。
`callerCapabilities`	object	SHOULD	调用方运行时能力。
`constraints`	object	MAY	延迟、授权、接口偏好、安全偏好等约束。
`candidateInterfaceRefs`	array	MAY	调用方从 Agent Description 中选出的候选 interface ID。
`candidateProtocols`	array	MAY	候选协议 artifact、Profile 或 URI；不应只使用自然语言大段文本作为唯一机器可处理内容。

7.4 `intent` 对象

`intent` 用于表达调用方希望完成的目标。它 SHOULD 包含结构化标签，也 MAY 包含自然语言描述。

```
{
  "name": "book_hotel_room",
  "description": "Book a hotel room for two people next Friday.",
  "intentTags": [
    "hotel.booking",
    "reservation.create"
  ],
  "inputSummary": {
    "city": "Hangzhou",
    "guests": 2,
    "date": "2026-07-03"
  }
}
```

调用方 SHOULD 遵循最小披露原则，不应在协商阶段提供完成业务所不必需的敏感数据。

7.5 `callerCapabilities` 对象

`callerCapabilities` 描述调用方当前支持的能力。常见字段包括：

字段	类型	说明
`supportedProfiles`	array	调用方支持的 ANP Profile。
`supportedSecurityProfiles`	array	调用方支持的安全模式。
`supportedContentTypes`	array	调用方可处理的内容类型。
`supportedExecutionModes`	array	调用方可接受的执行模式。
`limits`	object	调用方自身限制，例如最大消息大小、超时时间。

7.6 `constraints` 对象

`constraints` 用于表达调用方偏好或硬性约束。常见字段包括：

字段	类型	说明
`preferredInterfaceTypes`	array	偏好的接口类型，例如 `StructuredInterface`。
`requiresHumanAuthorization`	boolean	本次意图是否预期需要人类授权。该字段只是协商约束，不代表授权已完成。
`maxLatencyMs`	integer	期望最大延迟。
`allowNaturalLanguageFallback`	boolean	是否允许退回自然语言协议草拟或自然语言接口。
`requiredSecurityProfile`	string	必须使用的安全模式。
`preferredContentTypes`	array	偏好的内容类型。

8. Negotiation Result 对象

8.1 成功响应

`anp.negotiate` 成功时返回 `result`：

```
{
  "jsonrpc": "2.0",
  "id": "req-neg-001",
  "result": {
    "negotiationId": "neg-20260627-001",
    "status": "accepted",
    "selected": {
      "capability": "cap.hotel.booking",
      "interface": "interface.booking.structured.v1",
      "protocol": "openrpc",
      "profile": "anp.rpc.v1",
      "securityProfile": "transport-protected",
      "contentType": "application/json",
      "url": "https://grand-hotel.com/api/booking.openrpc.json"
    },
    "execution": {
      "mode": "direct_structured_call",
      "requiresHumanAuthorization": true,
      "timeoutMs": 3000
    },
    "schemas": {
      "requestSchema": "https://grand-hotel.com/schemas/create-booking-request.schema.json",
      "responseSchema": "https://grand-hotel.com/schemas/create-booking-
response.schema.json"
    },
    "validUntil": "2026-06-27T12:10:05Z",
    "negotiationDigest": "sha-256:BASE64URL_DIGEST"
  }
}
```

8.2 字段定义

字段	类型	要求	说明
`negotiationId`	string	MUST	协商结果标识。
`status`	string	MUST	`accepted`、`rejected` 或 `needs_more_information`。
`selected`	object	当 `accepted` 时 MUST	被选择的能力、接口、协议、Profile、安全模式、内容类型和 URL。
`execution`	object	SHOULD	后续执行模式和执行约束。
`schemas`	object	MAY	请求和响应 Schema。

字段	类型	要求	说明
`validUntil`	string	MAY	协商结果有效期。
`negotiationDigest`	string	SHOULD	协商结果摘要，用于缓存和复用。
`alternatives`	array	MAY	未被选中的候选方案。
`reason`	string	MAY	拒绝或需要更多信息时的人类可读说明。

8.3 `selected` 对象

`selected` 表示后续真正业务交互应使用的路径。常见字段包括：

字段	说明
`capability`	选中的能力 ID。
`interface`	选中的 Agent Description interface ID。
`protocol`	选中接口使用的协议，例如 `openrpc`、`ANP`、`MCP`。
`profile`	后续业务调用的 ANP Profile。
`securityProfile`	后续业务调用的安全模式。
`contentType`	后续业务负载内容类型。
`url`	后续业务接口或接口描述 URL。
`protocolArtifact`	可选协议 artifact URI 或 digest。

8.4 `execution` 对象

`execution.mode` 常见取值包括：

值	说明
`direct_structured_call`	直接调用结构化接口。
`direct_message`	使用私聊消息 Profile。
`group_message`	使用群消息 Profile。
`async_task`	创建异步任务或异步协作流程。
`stream`	使用流式交互。
`natural_language`	使用自然语言接口。
`natural_language_protocol_drafting`	使用自然语言草拟临时协议。

9. 自然语言协议草拟 fallback

结构化协商是本规范的主路径。自然语言协议草拟是 fallback，适用于：

- 目标 Agent 没有可满足意图的结构化接口；
- 调用方和目标方没有共同支持的 Profile 或 Schema；
- 双方需要临时生成实验性协议；
- 任务低频、一次性或高度个性化。

调用方可以在 `anp.negotiate` 请求中声明：

```
{
  "mode": "natural_language_protocol_drafting",
  "constraints": {
    "allowNaturalLanguageFallback": true
  }
}
```

自然语言 fallback MAY 产生一个协议 artifact 草案，但该 artifact 在被双方接受前不应被视为稳定协议。实现方 SHOULD 尽可能将自然语言草案转化为结构化字段、Schema、示例和测试用例。

10. 缓存、复用与协议 artifact

10.1 协商结果缓存

调用方 MAY 缓存 `NegotiationResult`，并在有效期内复用。缓存键 SHOULD 至少包含：

- 目标 Agent DID；
- 调用方 DID 或调用方能力摘要；
- intent 标签或规范化 intent 摘要；
- 选中的 interface ID；
- `selected.profile`；
- `selected.securityProfile`；
- `negotiationDigest`。

当出现以下情况时，调用方 SHOULD 重新协商：

- `validUntil` 过期；
- `anp.get_capabilities` 返回与缓存不一致；
- DID Document 或 `ANPMessageService` 发生变化；
- 后续业务调用返回不支持 Profile、接口、安全模式或内容类型；
- 本地策略、安全要求或用户授权状态变化。

10.2 协议 artifact

协议 artifact 是可复用的协议描述对象，MAY 包括：

- `protocol_id`；
- `protocol_uri`；
- `version`；
- 方法列表；
- 请求和响应 Schema；
- 支持的安全模式；
- 内容类型；
- 示例；
- 测试用例；
- digest；
- 签名或发布者信息。

示例：

```
{
  "protocol_id": "example.product-info.v1",
  "protocol_uri": "https://example.com/protocols/product-info/1.0",
  "version": "1.0",
  "description": "Get product information by product ID.",
  "depends_on_profiles": [
    "anp.core.binding.v1"
  ],
  "methods": [
    {
      "name": "product.get_info",
      "request_schema_uri": "https://example.com/schemas/product-info-request.json",
      "response_schema_uri": "https://example.com/schemas/product-info-response.json"
    }
  ],
  "content_types": [
    "application/json"
  ],
  "security_profiles": [
    "transport-protected"
  ],
  "digest": "sha-256:BASE64URL_DIGEST"
}
```

本规范不定义全局 artifact 注册表。artifact 的发布、审核、签名、版本治理和共识机制由未来规范定义。

10.3 与早期 0RTT 思路的关系

早期 ANP-06 中的 0RTT 设计通过握手消息携带 `usedProtocolHash` 来跳过协商。本规范不定义新的握手消息。等价能力应通过以下方式实现：

1. 调用方缓存 `NegotiationResult` 或协议 artifact；
2. 后续请求直接使用选中的 Profile / Interface / Schema；

3. 如果目标端点不支持该选择，返回明确错误；
4. 调用方根据错误重新调用 `anp.get_capabilities` 和 `anp.negotiate`。

11. 错误模型

`anp.negotiate` 使用 JSON-RPC `error` 对象返回错误。建议使用 ANP 自定义错误范围，并在 `error.data.anp_code` 中提供机器可识别错误码。

code	anp_code	含义
1600	`meta.negotiation_rejected`	目标拒绝协商请求。
1601	`meta.no_matching_interface`	没有满足 intent 和约束的接口。
1602	`meta.unsupported_negotiation_mode`	不支持请求的协商模式。
1603	`meta.unsupported_candidate_profile`	候选 Profile 不被支持。
1604	`meta.unsupported_security_profile`	候选安全模式不被支持。
1605	`meta.unsupported_content_type`	候选内容类型不被支持。
1606	`meta.more_information_required`	需要更多信息才能协商。
1607	`meta.authorization_required`	需要先完成身份认证或授权上下文。
1608	`meta.negotiation_expired`	协商结果或请求已过期。

错误示例：

```
{
  "jsonrpc": "2.0",
  "id": "req-neg-001",
  "error": {
    "code": 1601,
    "message": "No matching interface",
    "data": {
      "anp_code": "meta.no_matching_interface",
      "retryable": false,
      "details": {
        "unsupportedConstraints": [
          "requiredSecurityProfile"
        ]
      }
    }
  }
}
```

12. 安全与隐私考虑

12.1 身份认证

`anp.get_capabilities` MAY 作为匿名公共发现能力调用。

`anp.negotiate` 如果包含以下任一内容，调用方 SHOULD 提供 `auth.origin_proof` 或等价的 DID 认证上下文：

- `sender_did`；
- 敏感 intent；
- 用户上下文；
- 受限能力；
- 支付、预订、交易、授权、身份或隐私相关能力；
- 非公开接口候选；
- 服务端基于身份返回差异化协商结果的场景。

服务端 MAY 对匿名 `anp.negotiate` 返回能力子集、拒绝请求或要求认证。

12.2 协商结果不是授权

`NegotiationResult` 只说明"后续应该如何交互"。它不表示：

- 用户已经批准业务动作；
- 服务端已经授权访问受限资源；
- 支付、预订或交易已经成立；
- 调用方已经获得长期访问凭证；
- 后续业务请求可以省略该业务 Profile 要求的 proof。

如果被选择的接口要求 `humanAuthorization`，后续业务流程仍 MUST 按对应业务协议完成授权。

12.3 最小披露

调用方 SHOULD 最小化协商请求中的信息披露：

- intent 只提供协商所需摘要；
- 不在协商阶段提交完整支付信息、身份证件、私密消息或大对象内容；
- 对 `callerCapabilities` 做必要裁剪，避免暴露无关内部能力；
- 对协商日志进行脱敏处理。

12.4 降级攻击

调用方和服务端 MUST NOT 在未明确同意的情况下从更强安全模式降级到更弱安全模式。

如果调用方要求 `direct-e2ee`，服务端不能静默返回 `transport-protected`。如果无法满足要求，应返回明确错误。

12.5 协议 artifact 安全

如果协商结果引用外部协议 artifact，实现方 SHOULD 验证：

- artifact digest；
- 发布者或签名；
- Schema 完整性；
- 版本和依赖；
- artifact 中是否包含不安全代码、远程执行指令或过度权限要求。

本规范不要求执行远程代码。任何代码生成或代码加载都属于本地实现细节，必须在沙盒和最小权限环境中处理。

13. 旧版 PT=00 消息协商废弃说明

早期 ANP-06 草案曾定义在端到端加密消息负载内部增加 1 字节二进制头，用 `PT` 字段区分元协议、应用协议、自然语言协议和验证协议。该模型还定义了 `sourceHello`、`destinationHello`、`candidateProtocols`、`codeGeneration`、`testCasesNegotiation` 和 `fixErrorNegotiation` 等流程。

该早期模型已经废弃。新实现 MUST 使用本规范定义的 `anp.meta.negotiation.v1` 与 `anp.negotiate`，MUST NOT 实现或依赖旧的二进制 `PT` 消息头、旧 Hello 流程或旧代码生成流程作为 ANP-06 互操作路径。

本规范不提供旧模型到新模型的兼容映射，也不要求实现兼容网关。若某部署仍存在早期实验性实现，应将其视为私有历史实现，并通过独立迁移任务升级到本规范；该迁移不属于 ANP-06 最小互通要求。

14. 示例

14.1 Agent Description 片段

```
{
  "protocolType": "ANP",
  "protocolVersion": "1.1",
  "type": "AgentDescription",
  "url": "https://grand-hotel.com/agents/hotel-assistant/ad.json",
  "name": "Grand Hotel Assistant",
  "did": "did:wba:grand-hotel.com:service:hotel-assistant:e1_example",
  "securityDefinitions": {
    "didwba_sc": {
      "scheme": "didwba",
      "in": "header",
      "name": "Authorization"
    }
  },
  "security": "didwba_sc",
  "capabilities": [
    {
      "id": "cap.hotel.booking",
      "name": "Hotel room booking",
      "description": "Book hotel rooms, modify reservations, and check room availability.",
      "intentTags": [
        "hotel.booking",
```

```
        "reservation.create",
        "reservation.modify"
      ],
      "requiresHumanAuthorization": true
    }
  ],
  "interfaces": [
    {
      "id": "interface.negotiation.default",
      "type": "MetaProtocolInterface",
      "protocol": "ANP",
      "version": "1.0",
      "profile": "anp.meta.negotiation.v1",
      "binding": "jsonrpc-2.0",
      "url": "https://grand-hotel.com/anp",
      "methods": [
        "anp.get_capabilities",
        "anp.negotiate"
      ],
      "security": ["didwba_sc"],
      "securityProfiles": [
        "transport-protected"
      ],
      "negotiates": [
        "profiles",
        "interfaces",
        "schemas",
        "security_profiles",
        "content_types",
        "execution_modes"
      ],
      "description": "Endpoint for runtime capability and semantic meta-protocol
negotiation."
    },
    {
      "id": "interface.booking.structured.v1",
      "type": "StructuredInterface",
      "protocol": "openrpc",
      "version": "1.0",
      "profile": "anp.rpc.v1",
      "url": "https://grand-hotel.com/api/booking.openrpc.json",
      "capabilityRefs": [
        "cap.hotel.booking"
      ],
      "humanAuthorization": true,
      "description": "Structured booking interface for room reservation."
    },
    {
      "id": "interface.conversation.nl.v1",
      "type": "NaturalLanguageInterface",
      "protocol": "ANP",
      "version": "1.0",
      "profile": "anp.direct.base.v1",
```

```
      "url": "https://grand-hotel.com/anp",
      "capabilityRefs": [
        "cap.hotel.booking"
      ],
      "description": "Natural language conversation interface."
    }
  ]
}
```

说明：上例中的 `capabilities` 是 ANP-06 使用的可选扩展示例。实现方不应把它理解为 ANP-07 的必需顶层字段。

14.2 能力确认请求

```
{
  "jsonrpc": "2.0",
  "id": "req-cap-001",
  "method": "anp.get_capabilities",
  "params": {
    "meta": {
      "profile": "anp.core.binding.v1",
      "security_profile": "transport-protected",
      "operation_id": "op-cap-001",
      "created_at": "2026-06-27T12:00:00Z"
    },
    "body": {}
  }
}
```

14.3 能力确认响应片段

```
{
  "jsonrpc": "2.0",
  "id": "req-cap-001",
  "result": {
    "service_did": "did:wba:grand-hotel.com:e1_service",
    "supported_profiles": [
      "anp.core.binding.v1",
      "anp.meta.negotiation.v1",
      "anp.direct.base.v1",
      "anp.rpc.v1"
    ],
    "supported_security_profiles": [
      "transport-protected"
    ],
    "supported_content_types": [
      "application/json",
      "text/plain"
    ],
    "limits": {
      "max_request_bytes": "1048576"
```

```
    }
  }
}
```

14.4 协商请求与结果

协商请求和结果可参考第 7 节与第 8 节。实现方 SHOULD 在实际业务调用前保存 `negotiationId` 和 `negotiationDigest`，以便调试、缓存和审计。

15. 未来扩展

未来版本可以进一步定义：

- 标准协议 artifact JSON Schema；
- 协议 artifact 注册表和发现机制；
- 协商结果签名和可验证发布；
- 测试用例协商方法；
- issue report / fix negotiation 扩展；
- 与 MCP、OpenAPI、OpenRPC、A2A 等外部协议描述的映射；
- 多方协商和群协商；
- 协商隐私保护与最小披露 profile；
- 协议共识、投票、审核和治理机制。

版权声明

Copyright (c) 2024 ANP 开源社区
本文件依据 Apache License 2.0 发布，您可以自由使用和修改，但必须保留本版权声明。